



정산 표준화 솔루션 · 개발 상세 기록

정산 서비스 개발 상세 보고서

서비스화 · 계약/정산 · 회원·권한·보안 · 저장소 · 정식 배포
(v3.4 — 배포본 이후 전체)

작성일	2026년 8월 4일
작성자	최혜윤 (기획·PO 총괄)
연락처	rachel@hive-mind.kr
범위	배포본(v2) 이후 서비스 고도화 및 정식 배포 전 과정
불변식	431,296행 · 완전매칭 99.50% · 원본 대사 검증 86건 (전 과정 유지)

개요

배포본(v2)은 정산 데이터를 분석하는 도구였음. 개발본(v3)은 이를 출판사 단위 멀티테넌트 서비스로 전환하고, 계약 체계·정산 마감·회원/권한·데이터 격리·영속 저장소를 얻은 뒤 Railway 영속 볼륨으로 정식 배포했음.

영역	핵심
아키텍처	멀티테넌트 서비스화, 출판사↔레이블 계층, 저장소 SQLite 이전
계약·정산	계약 넘버링·상태·유효기간, 유통대행 지급, 정산 마감/원장
회원·권한·보안	RBAC·온보딩, 접속 게이트 분리, 서버측 테넌트 격리, 계약사 자체 관리
분석·운영	성과 분석, 수집 현황·이상 탐지·마감 스케줄, 회사 정보·삭제
배포·품질	Railway 영속 볼륨 정식 배포, 감사·회귀·리뷰·QA

1. 서비스화 전환 (v2 → v3)

정산 데이터 중심 화면을 출판사(테넌트)를 고객 단위로 두는 서비스 계층으로 전환. 출판사·계약·유통사 정보를 정산 데이터에서 파생하지 않고 영속 마스터로 분리 관리하고, 정산은 그 위에 얹는 구조로 재설계.

- 영속 마스터: 출판사·계약·유통사를 정산 세션과 독립된 마스터로 분리.
- 조회 계층은 read-only. 정산 매칭 엔진(core)은 손대지 않아 회귀 불변식 유지.
- 역할별 IA: 운영자 콘솔 ↔ 출판사 워크스페이스.
- core(엔진, 순수·무상태) ↔ service(마스터·워크플로) 모듈 경계. 모노레포 라이브러리 호출, 향후 HTTP 분리 대비.

설명 — 배포본은 정산서를 업로드해야만 화면이 동작하는 구조였음. 서비스로 성립하려면 정산 데이터가 없는 상태에서도 고객·계약·유통사를 관리할 수 있어야 함. 그래서 "데이터에서 파생"이라는 전제를 "영속 마스터가 먼저 있고 정산이 그 위에 붙는다"로 뒤집었음. 이때 정산 매칭 엔진을 순수·무상태(core)로 격리해 마스터·워크플로(service)와 분리한 것이 핵심으로, 엔진을 건드리지 않았기에 43만 행 회귀 불변식을 전 과정에서 유지할 수 있었고 이후 HTTP 마이크로서비스로의 분리도 열어두었음.

2. 출판사↔레이블 계층과 매핑

정산 데이터의 최소 단위는 레이블(임프린트)이고, 그 위에 출판사 덱스가 있음(예: 매드햇→스토린랩). 레이블을 출판사로 묶는 매핑을 구축.

- 본부장 제공 매핑표 시드 + 정규화 매칭(괄호·법인격·공백·대소문자 제거)으로 표기 변형 흡수.
- 미매핑 목록 2차 확인 → 통합 매핑표 458 레이블 → 158 출판사. 테넌트 843 → 128곳.
- 미상 레이블은 데이터로 역추적(동일 작품이 이미 매핑된 레이블에 있는지). 작가 교차는 노이즈라 배제.

에피루스 사례 — 미매핑이 곧 매칭 실패는 아님

미매핑 824건 중 586건이 실은 책 제목이었음. 유통사 에피루스 정산서에 출판사 열이 없어 파서가 4번 열(작품 부제)을 레이블로 읽은 것. 본부장 매핑상 에피루스=블루마운틴소프트 단일 채널이므로 파서에 상수 레이블을 지정해 교정.

```
dict(platform="에피루스", ... columns_pos=dict(title=2, author=3, gross=6, settle=8),
      label_const="에피루스") # 4번 열(부제) 제외, 유통사=단일 레이블 고정
```

설명 — 미매핑 수치(824)만 보면 매칭 알고리즘 실패로 오해하기 쉬움. 실제 값을 열어보면 원본 포맷 차이로 엉뚱한 열이 레이블 자리에 들어온 것이 원인이었음. 유통사마다 정산서 열 구성이 다르고, 특히 출판사 열이 없는 유통사(에피루스)는 부제·세트명 등이 레이블로 오인식됨. 파서 스펙에 유통사별 열 위치와 상수 레이블을 명시하는 방식으로 근본 교정했고, 교정 후에도 431,296/99.50%/86 불변임을 확인해 데이터 왜곡이 없음을 검증했음.

3. 저장소 이전 — SQLite

마스터(출판사·계약·회원·초대·플래그·별칭·유통사 설정 등)를 JSON 파일에서 SQLite(WAL)로 이전.

- `db.py` 문서-블록 저장소. 컬렉션 전체를 하나의 값으로 트랜잭션 교체(원자적).
- 최초 기동 시 기존 JSON을 1회 자동 이관. `load_*/save_*` 시그니처를 그대로 유지해 호출부 무변경.
- WAL·busy_timeout으로 동시 읽기/쓰기 안정화. 검증: 이관(출판사 128·회원 등)·라운드트립·회귀 유지.

설명 — JSON 파일 저장은 세 가지 실제 위험을 안고 있었음. ① 프로세스 재시작 시 컨테이너 파일시스템이 초기화되면 데이터 소멸, ② 동시 쓰기 시 나중 쓰기가 앞 쓰기를 덮어쓰(clobber), ③ 쓰기 도중 크래시 시 파일이 빈 채로 남음 — 가상 파일럿 중 계약이 4건에서 0건으로 사라지는 현상이 실제로 관측되었음. SQLite(WAL)는 트랜잭션·durability로 이 세 가지를 구조적으로 해결함. 다만 저장소를 통째로 바꾸면 호출부 전체가 흔들리므로, `db.py`를 문서-블록(컬렉션=하나의 JSON 값) 형태로 설계해 기존 `load/save` 시그니처를 100% 보존했음. 그 결과 엔진·API 코드는 한 줄도 바꾸지 않고 저장 계층만 교체할 수 있었음.

4. 계약 체계

4.1 계약 넘버링

계약마다 고유번호를 부여. 번호만으로 출판사·원작·매체·권리·형태가 읽힘.

```
0001 - AA001 - N - 1 - 00
|      |      |      |      |
|      |      |      |      |  콘텐츠유형(웹소설 00 / 웹툰 01 / 번역 02 ...)
|      |      |      |      |  계약유형(1 배타적발행권 / 2 2차적저작물 / 3 외주)
|      |      |      |      |  웹소설(N) / 웹툰(T)
|      |      |      |      |  콘텐츠 고유번호(원작 1개=하나, 파생은 공유)
|      |      |      |      |  출판사 구분값(첫 계약 시 자동 0001~)
```

파생 계약(웹툰화·외주)은 원작의 콘텐츠번호를 공유하고 뒤 세 자리로 구분: `0001-AA001-N-2-01` (웹툰화 2차저작물).

4.2 상태 · 유효기간

요소	정책·동작
상태(초안/확정/종료)	소급 일괄=초안. 초안은 예상 지급액 미반영 → 확정 후 반영(미검토 기본값 부풀림 방지)
유효기간(시작/종료월)	해당 기간 정산행에만 배분을 적용. 서명 이전 과거 소급 방지

4.3 소급 일괄 등록 (온보딩)

출판사 합류 시 기존 계약을 한꺼번에 등록. 이미 올라온 정산 데이터의 작품·작가를 계약 초안 소스로 재활용 → 미계약 작품 목록에서 선택 → 배타적발행권 초안 일괄 생성(콘텐츠번호 자동 연번). 이후 배분율·계약서를 개별 보정.

설명 — 실제 온보딩 시나리오는 신규 계약 입력이 아니라 이미 존재하는 계약을 소급 등록하는 것임. 출판사 한 곳이 수백 개 작품을 갖고 있으므로 건별 입력은 비현실적임. 그래서 이미 보유한 정산 데이터(작품·작가·레이블)를 계약 초안의 소스로 뒤집어, 미계약 작품을 자동 식별해 초안을 일괄 생성함. 상태를 초안으로 두고 예상 지급액에 반영하지 않는 이유는, 검토되지 않은 기본 배분율이 지급 예측을 부풀리는 것을 막기 위해서임. 담당자가 배분율을 확정해야 비로소 지급 예측에 들어감.

5. 정산 산식

작가 지급 = 유통사 제공 정산액 × 계약 배분율 - 선인세(MG) - 원천징수. (원천 인세엔진이 아니라 유통사 제공값 기준.)

5.1 작가 지급 vs 출판사 지급 (유통대행 3계층)

"작가 - 출판사 - 관리자(하이버마인드)" 3계층. 유통대행 관점에서는 출판사를 작가처럼 취급(지급처=출판사). 계약 메커니즘은 그대로 두고 매칭 키만 작가명 → 출판사명(레이블→출판사 해석·산하 레이블 합산)으로 전환.

```
payee_type=="출판사": rows = 레이블이 그 출판사로 해석되는 정산행
                        base = Σ settle · payout = base × 배분율 - MG - 세
```

5.2 작품 범위 · 중복 경고

- 여러 작품 = OR(하나라도 포함). 포함매칭이라 권차·외전 자동 포함.
- 같은 지급대상 계약이 실제 매칭 정산행 기준으로 겹치면 이중 배분 경고(문자열이 아닌 행 교집합 판정 → 오탐 없음).

5.3 MG 누적 · 명세서 통일

- MG를 로드 기간 누적에서 회수하고 미회수분(잔액)을 표시(이월 대상).
- 작가 명세서 PDF를 확정 계약의 배분율·MG·세율로 통일 → 계약관리 예상지급액과 값 일치(0.033 하드코딩·별도 요율 제거).

설명 — 정산 산식의 기준은 정가×판매량×인세율의 원천 계산이 아니라, 유통사가 제공한 정산액임. 유통사가 이미 자사 수수료를 댄 값을 주므로, 하이버마인드는 그 값에 계약 배분율을 적용하고 선인세·원천징수를 차감하는 2단계 구조를 취함. 유통대행 계층에서는 지급 대상이 출판사가 되는데, 이때 새 계산을 만들지 않고 매칭 키를 작가명에서 출판사명(산하 레이블 합산)으로만 바꿔 동일한 계약 메커니즘을 재사용했음. 계산식 자체를 바꾸는 변경(명세서 산식 통일 등)은 회귀 위험이 크므로, 관련 정책 4건(초안/확정·유효기간·MG 모델·명세서 산식)을 먼저 확정된 뒤 반영했고 반영 후에도 불변식을 유지했음.

6. 정산 마감 / 원장

월 지급액을 확정=불변 스냅샷으로 고정하는 정산 마감 기능. 초안 → 확정 → 지급 3단계. 선인세(MG)는 이월 원장으로 관리해 이중 차감을 방지.

- 확정 시점의 계약별 지급 라인을 스냅샷으로 고정 → 이후 데이터·계약이 바뀌어도 확정본은 불변.
- MG 회수는 이전 확정 런에서 이미 회수한 금액을 승계 → 다음 런은 잔액만 상계.

런1: MG 5천만 상계 → 지급 21,990,569

확정 후 런2: MG 이월(추가 상계 0) → 지급 70,340,569 # 이중 차감 없음

설명 — 정산 시스템이 분석 도구와 구분되는 지점이 바로 이 기능임. 분석 도구는 매번 최신 데이터로 재계산한 추정치를 보여주면 되지만, 실제 지급의 근거가 되는 금액은 특정 시점에 확정되어 이후 변하지 않아야 함(재현 가능성·감사 대응). 그래서 확정 시점의 지급 라인 전체를 스냅샷으로 박제함. 특히 선인세(MG)는 여러 달에 걸쳐 회수되므로, 지난 달에 이미 회수한 금액을 다음 달이 모르면 이중으로 상계되는 사고가 남. 이를 막기 위해 확정 런이 회수한 MG를 원장에 기록하고 다음 런이 잔액만 상계하도록 설계했으며, 실제 2개 런으로 이중 차감이 없음을 검증했음.

7. 회원 · 권한 · 온보딩

7.1 역할·권한(RBAC)

조직	역할	요지
운영사	최고관리자 / 운영자 / 뷰어	전 기능·승인 / 운영·계정발급 / 조회
계약사	출판사관리자 / 담당자	자사 관리·사용자 관리 / 자사 조회

권한 키(resource:action)를 역할에 매핑. 원칙(sian ERP 참고): 활성 상태만 로그인, 최고관리자 바이패스, 동의 전 차단.

7.2 흐름

- 운영사 가입·승인: 최초 가입자=최고관리자 자동(부트스트랩), 이후 승인 필요.
- 계약사 계정 발급: 운영자가 이메일 초대 → 링크 → 이름·비번 설정 → 활성. 첫 계정=출판사관리자.
- 최초 접속: 동의 화면 → 담당자 정보 입력(온보딩) → 워크스페이스.

7.3 브랜드 표기

계약사 화면은 계약사명이 주 브랜드, 하이브마인드는 제공처로 표기(로그인 조직에 따라 자동 전환).

8. 접속 게이트 분리 · 계약사 자체 계약 관리

운영사와 계약사의 진입 경로를 분리하고, 계약사가 자사 계약·정산 마감을 직접 관리하도록 개방.

- **접속 게이트 분리**: 운영사=회원가입 후 승인 / 계약사=초대 링크 발급. 초대는 계약사 계정 전용(운영사 권한 상승 차단).
- **계약사 자체 관리**: 신설 권한 `contracts:manage_own` — 출판사관리자가 자사 계약·정산 마감을 직접 관리(담당자는 조회 전용).

자사 스코프 강제

권한을 여는 동시에 계약·마감 쓰기 지점(생성·수정·삭제·첨부·일괄·마감 생성/확정/삭제, 총 10곳)에 소속 검사를 적용. 운영자는 전 테넌트, 계약사는 자사만 쓰기 가능.

```

CONTRACT_WRITE = ("contracts:manage", "contracts:manage_own") # require는 OR
require(request, *CONTRACT_WRITE)
if not can_access_tenant(request, tenant_id): # 운영자=전체 True, 계약사=자사만
    raise HTTPException(403)

```

검증(라이브)	결과
계약사 → 자사 계약 생성/삭제	200
계약사 → 타 출판사 계약 생성/수정/삭제	403
계약사 → 전사 마감 현황(close_status)	403
운영자 → 전 테넌트 계약 관리	정상 유지

설명 — 계약의 당사자는 출판사이므로, 자사 계약·정산은 출판사가 스스로 관리하는 것이 자연스러움(대행이 아닌 셀프서비스 모델). 이를 열려면 두 가지를 동시에 처리해야 했음. 첫째, 새 권한 contracts:manage_own을 신설해 출판사관리자에게만 부여했음(담당자는 조회 전용 유지). 둘째, 권한만 열면 계약사가 파라미터의 출판사 ID를 바꿔 타사 계약을 만들 수 있으므로, 계약·마감 쓰기 10개 지점 전부에 소속 검사를 추가했음. 권한 검사(require)가 OR로 동작하는 점을 이용해, 운영자 전용이어야 하는 전사 마감 현황(close_status)에는 manage_own을 포함하지 않아 계약사에게 다른 출판사의 마감 상태가 노출되지 않도록 차단했음. 라이브에서 6종 시나리오로 자사 허용·타사 차단을 검증했음.

9. 보안 — 서버측 테넌트 격리

설계 리뷰 최우선 지적: 권한 검사가 회원·마스터 쓰기에만 있고, 정산 조회·내보내기·업로드·매칭은 로그인+활성만 확인했음. 스코프를 클라이언트가 넘기는 값에 의존해, 계약사가 파라미터만 바꾸면 타사 매출·계약·계좌·정산서를 열람할 수 있었음.

- 정산 조회/내보내기/명세서를 로그인 사용자의 테넌트 레이블로 서버측 강제(클라이언트 스코프 무시, 라벨 교집합).
- 계약·유통사·첨부는 소속 테넌트 검사(타사 403). 첨부 다운로드 IDOR 차단.
- 전역 매칭/연구소 변경 POST는 운영자 전용 미들웨어 가드.

검증	결과
같은 세션 · 계약사 summary	자사 6.04억만
같은 세션 · 운영자 summary	전체 11.97억
계약사 → 타사 계약/유통사/첨부	403
계약사 → 라벨 인젝션(타사 강제)	0건(교집합 차단)

설명 — 화면에서 보이지 않는 것과 접근할 수 없는 것은 다름. 클라이언트가 보내는 값(테넌트 스코프)은 조작 가능하므로, 권한과 스코프는 반드시 서버에서 강제해야 함. RBAC 모델이 정의되어 있어도 각 엔드포인트에 연결되지 않으면 무의미함. 이 격리 계층은 8장의 계약사 자체 관리와 짝을 이룸 — 자사 관리를 개방하려면 그 전제로 서버측 테넌트 경계가 완결되어 있어야 하기 때문임.

10. 데이터 분석 · 운영 자동화

10.1 성과 분석

월별 추세·전월 대비(MoM)·유통사 성과(정산액·비중·추정 정산율)·상위 작가/작품 파레토·집중도·유통대행 비중. 계약사는 자사 스코프, 운영자는 전체.

10.2 운영 자동화

- 수집 현황판: 유통사 31곳 × 월 매트릭스(검증/미검증/미수신).
- 이상 탐지: 전월 대비 급변 $\pm 40\%$ · 유통사 누락 · 음수 · 합계 불일치 · 파싱 오류.
- 마감 스케줄러: 출판사별 마감 현황, 마감일·대상월·리마인더 설정, 백그라운드 감사 리마인더.

설명 — 운영의 최대 병목은 유통사 31곳의 정산서가 언제 도착했고 검증됐는지 추적하는 일, 값의 이상(급변·누락·음수·합계 불일치)을 마감 전에 잡는 일, 그리고 마감 기한을 놓치지 않는 일임. 세 가지 모두 유통사 외부 시스템 접근 없이 이미 수집된 데이터만으로 구현 가능하므로 우선 자체 구현했음. 유통사 정산서 자동 수집(RPA)은 유통사별 로그인·다운로드 수단이 필요해 별도 과제로 분리했음.

11. 회사 정보 자가 관리 · 삭제 · 홈 재편

11.1 회사 정보 · 삭제

계약사가 사업자번호·대표자·주소·정산 입금계좌·세금계산서 이메일을 직접 입력·관리. 회원·출판사 삭제(자사만 수정, 마지막 최고관리자·본인 삭제 방지).

11.2 운영자 콘솔 홈 재편

정산 요약 홈에 고객사(출판사)별 정산 블록을 1차로 배치(행 클릭 시 해당 고객사 화면으로 이동), 유통사별·상위 작가는 2차로 이동. 운영자 역할 토글 라벨을 "운영자 콘솔 / 고객사 화면"으로 명확화.

설명 — 운영사의 고객은 128개 출판사임. 정산 요약 홈이 v2 분석 도구 시절의 유통사 중심 구성을 유지하고 있어, 운영사 정체성(고객=출판사)에 맞게 고객사별 정산을 첫 화면 1차 블록으로 올렸음. 반면 계약사(출판사) 워크스페이스는 "내 정산이 리디·카카오별로 얼마"가 핵심이므로 유통사 중심 구성을 그대로 두었음 — 같은 화면이라도 보는 주체에 따라 우선 축이 다름.

12. 정식 배포 — Railway 영속 볼륨

모든 상태(SQLite DB·세션·계약 첨부)를 영속 볼륨에 저장해, 재배포에도 계정·데이터가 보존되는 상시 운영 상태로 배포.

- 볼륨 마운트에 DB·첨부 경로 고정, 환경변수 분리(세션 시크릿·DB 경로·첨부 경로·API 키) — 시크릿은 코드·저장소 미포함.
- 최초 시드: 운영사 최고관리자 · 정산 1~4월(431,296행) · 출판사 128곳 · 계약사 데모 1곳.
- 재배포 후 데이터·세션 유지 확인.

설명 — 3장의 SQLite 이전이 데이터 안전성의 절반이라면, 영속 볼륨은 나머지 절반임. 컨테이너의 기본 파일시스템은 재배포마다 초기화되므로, DB 파일이 볼륨 위에 있지 않으면 매 배포마다 계정·계약·정산이 사라짐. 볼륨을 마운트하고 DB·첨부 경로를 그 위로 지정한 뒤, 세션 시크릿을 고정값으로 분리해 재배포 후에도 기존 로그인 세션이 유지되도록 했음. 시크릿은 환경변수로만 주입하고 코드·저장소에 두지 않음.

13. 운영 · 품질

- 초대 만료(14일)·비밀번호 8자·상태값 검증·마지막 최고관리자 정지 방지(자기잠금 차단).
- 비밀번호 재설정(본인 변경 + 관리자 임시 초기화), 초대 취소/재발송.
- 감사 로그: 로그인·가입·승인·권한·상태·초대·비번 이벤트 기록·조회.
- 모든 정산 영향 변경 후 회귀 불변식 재확인(431,296/99.50%/86) — 전 과정 및 배포본에서 유지.
- 설계 리뷰 2건으로 갭·정책·보안 도출(비이슈 2건도 반증해 과도한 수정 방지), 전체 기능 QA 테스트 시트(8시트 90케이스) 작성.

14. 성과 요약 · 남은 과제

지표	배포본(v2)	개발본(v3.4)
성격	정산 분석 도구	멀티테넌트 정산 서비스(정식 배포)
출판사·계약·유통사	정산 데이터 파생	영속 마스터(분리)
진입	정산 업로드 후	업로드 없이 관리
사용자	단일 접근	회원·역할·권한(RBAC)
데이터 격리	없음	서버측 테넌트 격리
계약·정산	요율 직접	넘버링·상태·유효기간·유통대행·정산 마감/원장
계약사 관리	—	자사 계약·마감 자체 관리(자사 스코프 강제)
저장·운영	JSON	SQLite · Railway 영속 볼륨(상시 운영)
회귀 불변식	431,296/99.50%/86	동일(유지)

남은 과제

- 정식 오픈 전: 로그인 시도 제한(레이트리밋) · SMTP(초대·알림 메일 실발송).
- 후속: 작가 마스터+명세서 자동 배포(SMTP 후) · 세무 연동(팝빌 등, 파트너 확정 후) · 자동 마감 생성 · 페이지네이션·계약 일괄 편집 · 공개 API/웹훅.
- 정책 확인(본부장): 원천징수율(현 3.3%) · 약관 · 마감 후 정정 정책.

개발본은 로그인·권한·온보딩·데이터 격리·계약 신뢰성·정산 마감·계약사 자체 관리·영속 저장소를 갖추고 Railway 영속 볼륨으로 정식 배포된 상태로, 외부 계약사 계정 운영이 가능한 최소 안전선을 통과했음.